

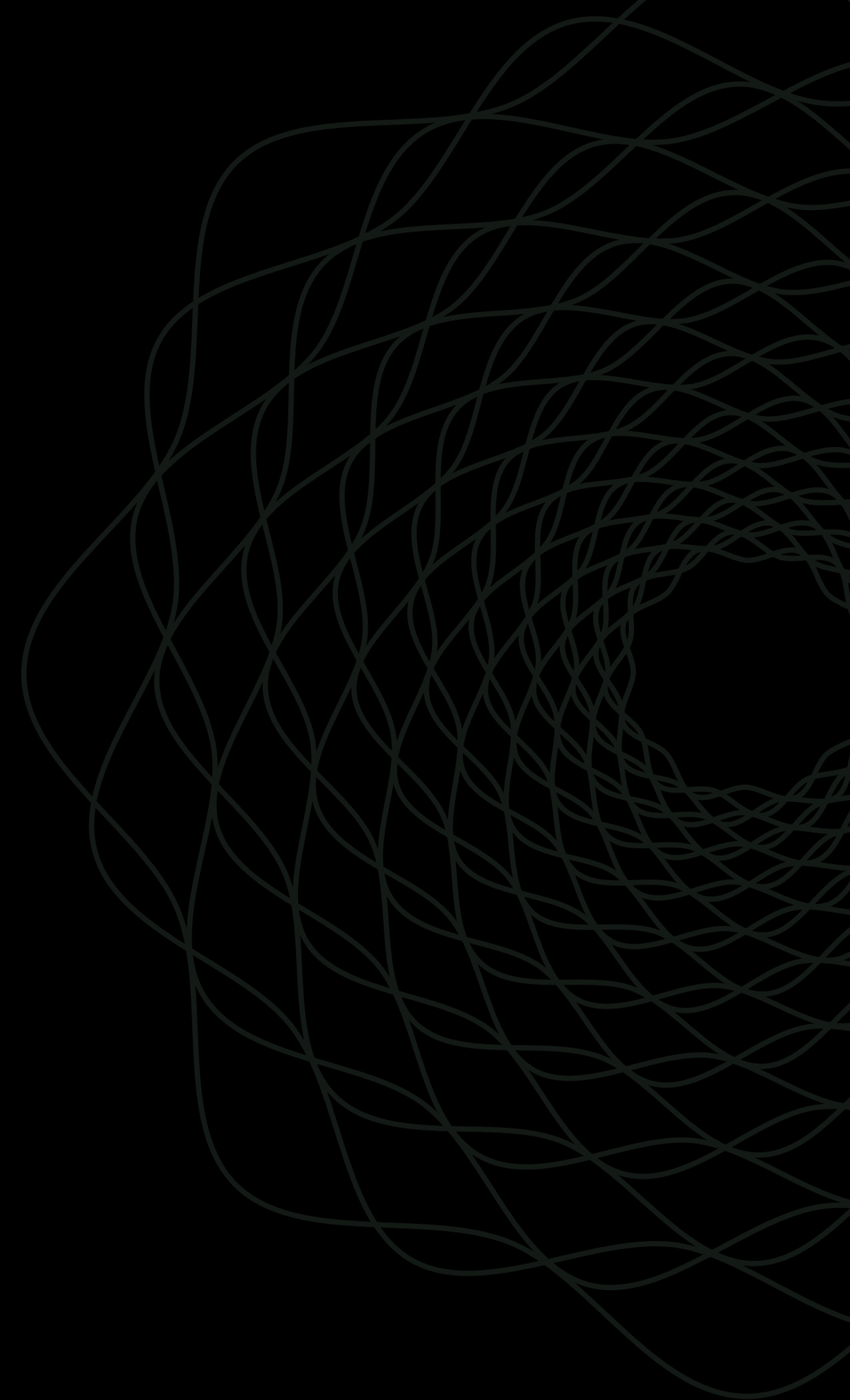


Weather App Case Study

Presented by: Shivam Namdeo

Abstract

1. Case Study
2. Problem Statement
3. Solution Approach
4. Technology Stack
5. Design Patterns
6. System Design Principles
7. NFR Coverage
8. 12-Factor App Alignment
9. CI/CD Diagram



1. Case Study

Designed a Weather Prediction providing 3-day forecasts for any city.

1. Build, test, and deploy a Weather Prediction Microservice
 - a. Input: City name → Output: Next 3 days forecast
2. Add intelligent alerts:
 - a. Rain → Carry umbrella
 - b. Temp > 40°C → Use sunscreen lotion
 - c. Wind > 10mph → Too windy, watch out!
 - d. Thunderstorm → Don't step out!
3. Must work online + offline (fallback when API is down)
4. Must follow SOLID, 12-Factor, HATEOAS, TDD, BDD, NFR

2. Problem Statement

1. Weather App aims to serve globally 200k cities.
2. OpenWeather API has strict rate limits based on subscription, so repeated calls quickly exhaust the quota.
3. Users expect fast responses for many cities, but cache-aside TTL creates challenges:
 - a. If TTL is too low (e.g., 20 mins): Many cache expiries → Redis becomes empty → sudden burst of API calls → API quota gets exhausted.
 - b. If TTL is too high: System shows stale/outdated weather data, reducing accuracy.
4. System must support high-traffic, low-latency, and also work offline using cached data when API is down.
5. Adding new prediction rules should require minimal code change, and entire solution must be production-ready (secure secrets, CI/CD, Docker, scalable design).

3. Solution Approach

1. Designed a 2-service architecture: **Weather-SVC** (Openweather API fetch + Adding Prediction + Rate Limitor 150 req/min) and **Weather-Cache** (Redis caching + On Demand + Scheduler Refresh logic).
2. Weather-Cache service handles cache hit/miss logic, on-demand refresh, last-access checks, last-refresh checks.
3. Built a multi-region Redis cache (6 regions) to support 200k global cities, ensuring low latency & high availability.(Multi-Region is **conceptual implemented** and deployed on one region only)
4. Used a hybrid refresh model: **On-demand refresh + 15-min scheduled refresh to keep Redis updated without API overload.**
5. Implemented HOT/MEDIUM/LOW city segmentation to optimize memory (2 GB global) and maintain **~90% cache-hit rate.**
 - a. **city segmentation:**
 - i. **HOT (≥ 140 hits $\rightarrow$ refresh every 20 min),**
 - ii. **MEDIUM (≥ 70 hits $\rightarrow$ refresh every 30 min),**
 - iii. **LOW (idle $\rightarrow$ evict after 1 hour).**
6. Ensured smooth fallback using offline mode, serving last 1-hour cache data when OpenWeather API is down.
7. Full data flow: UI $\rightarrow$ Weather-Cache $\rightarrow$ (Redis / Weather-SVC) $\rightarrow$ Response, ensuring fast, consistent results.
8. Added Swagger docs, error codes, logging, and health endpoints for production readiness.

4. Tech Stack

Frontend

- Next.js (TypeScript)
- TailwindCSS
- API abstraction layer
- Loading/error states
- Retry + Timeout logic
- React-Testing lib

Backend

- Java 21 + Spring Boot
- Redis(LRU,LFU)
- RestTemplate timeouts
- Spring Scheduler + Virtual Threads
- Global Exception Handling
- Sliding Window Rate Limiter
- TDD, BDD Testing

Infrastructure

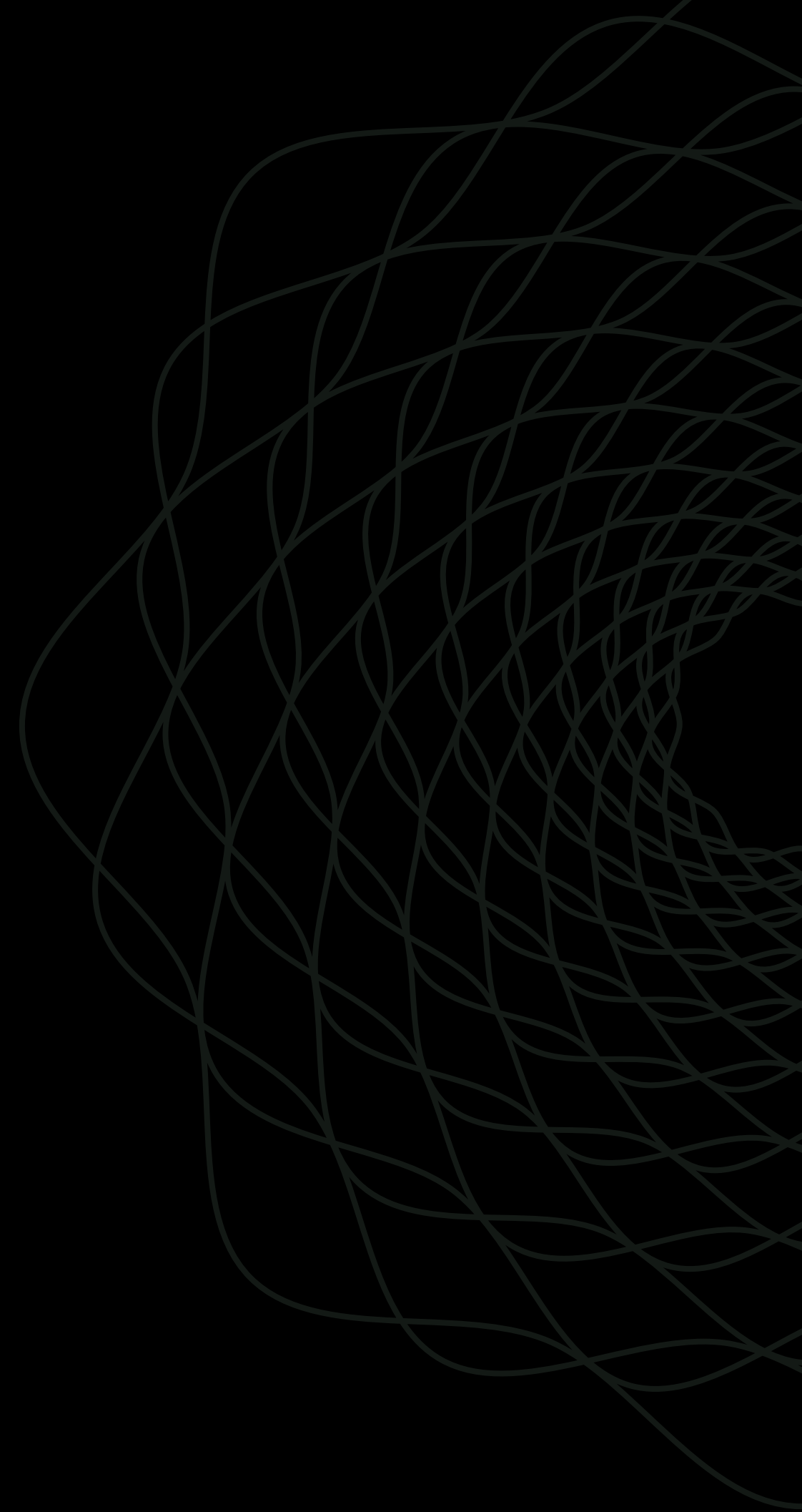
- Docker / Podman
- Docker Compose
- Azure AKS / Kubernetes
- GitHub Actions
- Prometheus, Kubernetes Logs
- Jmeter for load testing

External API

OpenWeather
Forecast API (3-day
weather data).

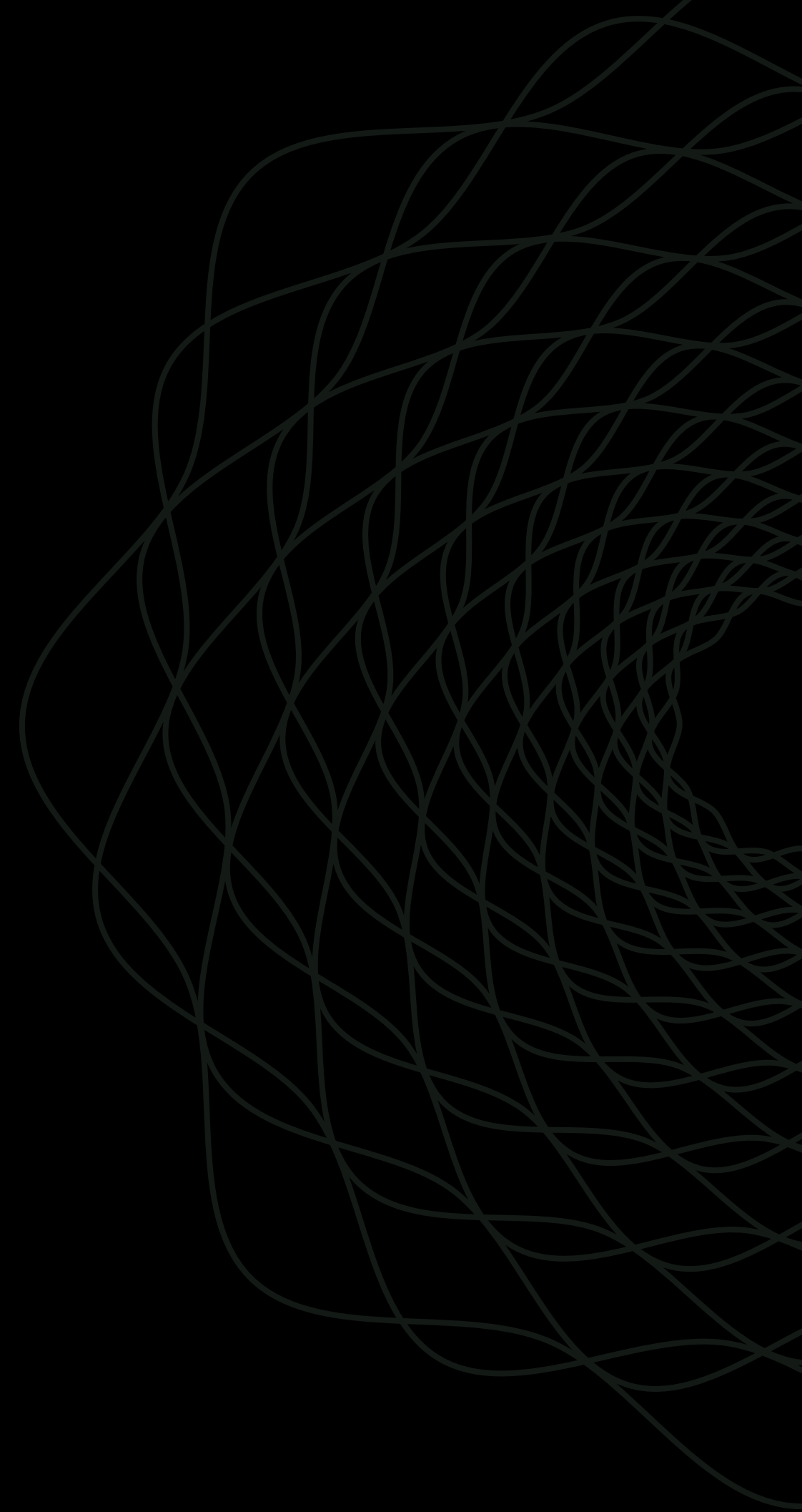
5. Design Patterns

- Cache-Aside Pattern
 - Fast reads from Redis; fallback to Weather-SVC on cache miss.
- Strategy Pattern
 - Dynamically decides HOT / MEDIUM / LOW refresh behavior.
- Factory Pattern
 - Produces correct refresh strategy based on hit-count.
- Template Method Pattern
 - Shared skeleton for refresh logic (staggering, async flow, updates).
- Facade Pattern
 - Weather-Cache service hides internal logic of SVC + Redis + Scheduler.
- Service Pattern
 - Clear abstraction for Redis operations.
- Singleton Pattern
 - Global rate limiter and scheduler instances.
- Inflight Request Pattern (Future Aspects)
 - Deduplicates parallel refresh calls for the same city.



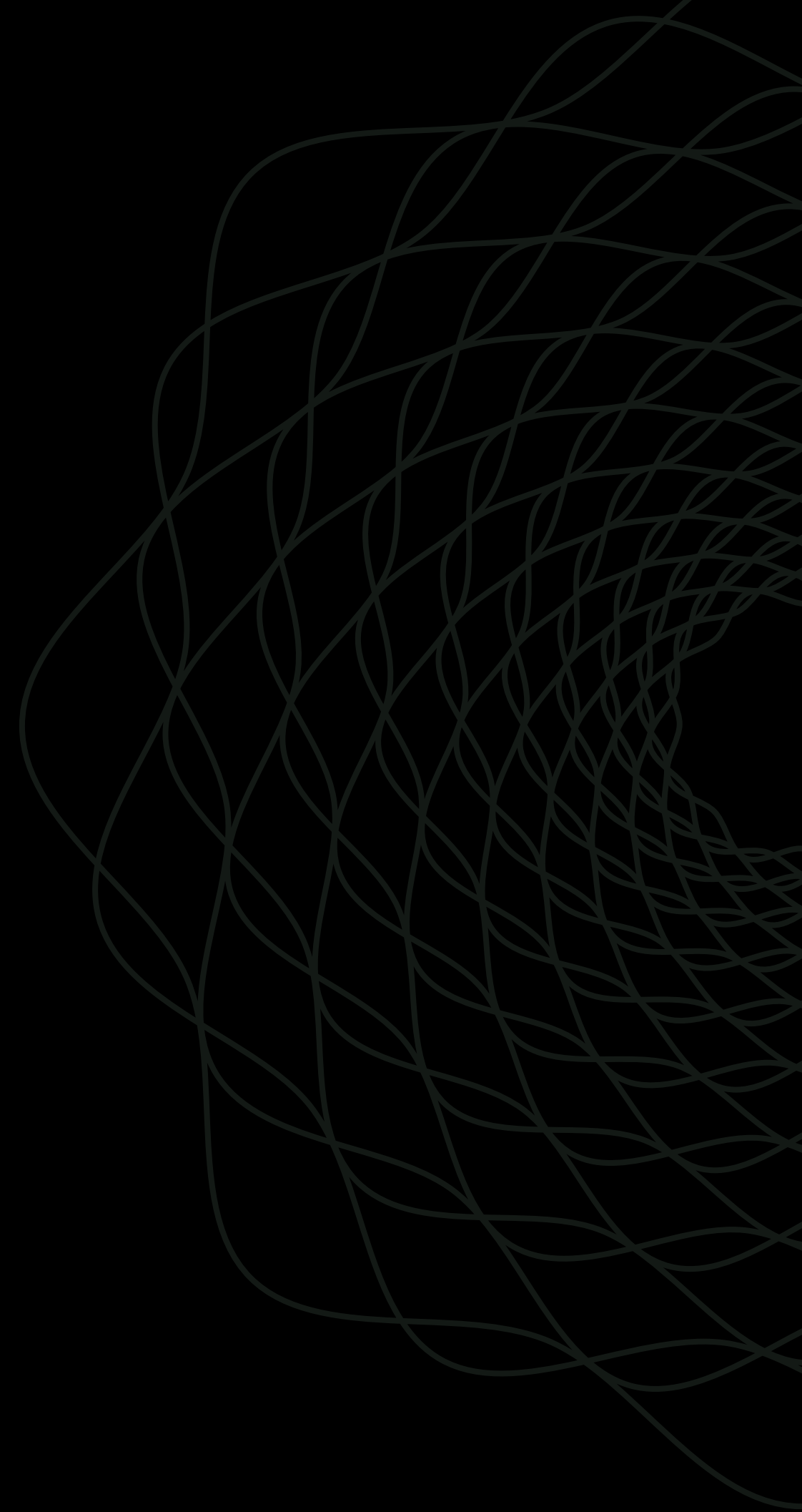
6. System Design Principles

- BASE Model (Eventually Consistent)
 - Data refresh happens asynchronously; users always get fast reads.
- 12-Factor App Adherence
 - Cloud-native, scalable, minimal configuration drift.
- SOLID / DRY / KISS
 - Clean, maintainable, modular codebase.
- Stateless Services
 - Enables horizontal autoscaling across regions.
- Regional Isolation
 - Each region runs independently for low latency.
- Horizontal Scalability
 - Stateless pods + Redis caching enables linear scale-out.
- Observability-First
 - Metrics, health checks, structured logs.
- Resilience & Fault Tolerance
 - Retries, timeouts, backoff, fallback-to-cache mode



7. NFR Coverage

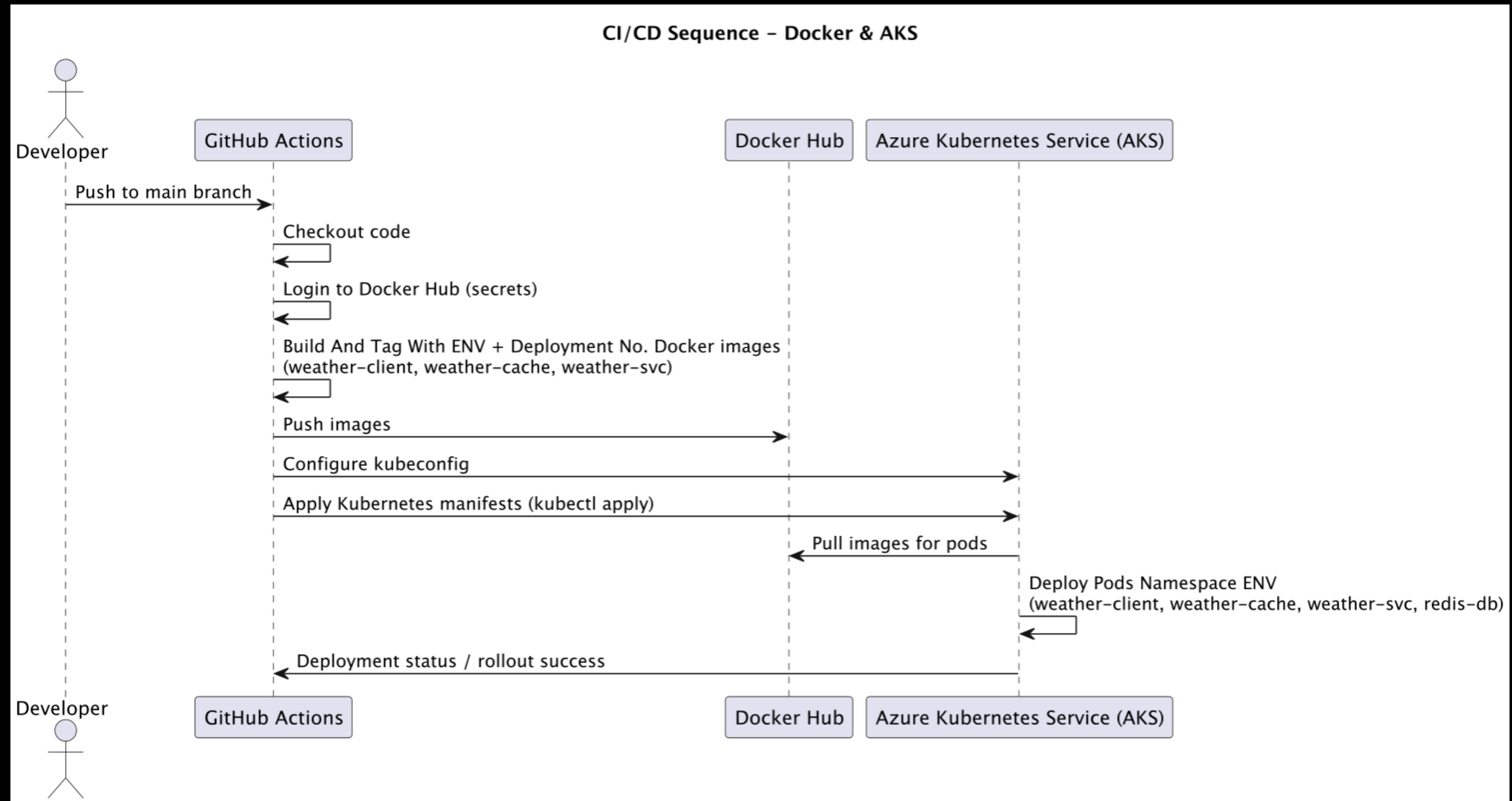
- Performance
 - Redis caching, sub-10ms hits, virtual-thread scheduler.
- Scalability
 - Stateless microservices + Kubernetes autoscaling.
- Availability
 - Multi-region setup + offline cached responses.
- Low Latency
 - <10ms cache hits, <1s cache miss path.
- Fault Tolerance
 - Scheduler staggering, cached fallback, regional isolation.
- Resilience
 - Timeout, retry, fallback, error boundaries.
- Security
 - API keys in env vars, CORS restrictions, secrets in Docker/K8s and in Github Actions
- Observability
 - Spring Actuator, Prometheus metrics, structured logs.
- Cost Optimization
 - 90% fewer external API calls due to caching + tiering.



8. 12-Factor App Alignment

- 1. Codebase:
 - Single repo with clear modular boundaries.
- 2. Dependencies:
 - Managed by Maven (BE) & npm (FE).
- 3. Config:
 - All configs injected through .env or K8s secrets, Github Action also taking from Github Secrets
- 4. Backing Services:
 - Redis + OpenWeather used as attached services.
- 5. Build/Release/Run:
 - Docker image → Compose → AKS deployment.
- 6. Processes:
 - Stateless containers, zero shared memory.
- 7. Port Binding:
 - Services explicitly expose 8080 / 8081 / 3001.
- 8. Concurrency:
 - Horizontal scaling via Kubernetes HPA, using virtual thread with scheduler refresh.
- 9. Disposability:
 - Fast startup + graceful shutdown for scheduler/SVC.
- 10. Dev/Prod Parity:
 - Same images; only env vars differ.
- 11. Logs:
 - STDOUT logging - Loki/Grafana compatible.
- 12. Admin Processes:
 - Scheduler runs as a background process inside Weather-Cache service.

9. CI/CD Diagram





Thank You